MODELSCOPE AI

AI Model Evaluation & Observability Platform

CODING AGENT IMPLEMENTATION SPECIFICATION

Product requirements • System architecture • Repository structure • Implementation sequence • Quality gates

Project scope	Full-featured platform, delivered incrementally
Frontend	Angular + Tailwind CSS
Backend	Python + Django REST Framework
Data and jobs	PostgreSQL + Redis + Celery
AI integrations	Cloud APIs, Hugging Face, LM Studio and Ollama
Evaluation and telemetry	DeepEval + OpenTelemetry
Development environment	Docker Compose

Version 1.0 | 8 November 2026 | Baseline instructions for an AI coding agent

1. Mission and operating instructions

Your role: Act as a senior full-stack engineer and technical lead. Build ModelScope AI as a maintainable, secure, tested application. Follow this specification as the source of truth. Do not replace major architectural choices without explaining the reason and recording the change.

Product mission: Give software developers and AI engineers one workspace in which they can configure AI providers, run comparable model experiments, evaluate outputs, inspect execution traces, monitor operational metrics and perform practical AI security checks.

This is a full-scope product, but implementation must be incremental. Build a working vertical slice at each milestone. Do not attempt to generate the entire application in one pass.

The initial architecture is a **modular monolith** with asynchronous workers. Do not introduce microservices, Kubernetes, a separate vector database or complex event streaming unless a documented requirement justifies it.

Use precise, honest product language. Never invent token counts, provider capabilities, cost rates, evaluation scores or telemetry. Represent unavailable measurements as null/unknown and explain why.

Non-negotiable agent rules

- Inspect the existing repository before changing it. Preserve working code and existing conventions where they do not conflict with this specification.
- If the repository is empty, initialise the agreed structure. If it contains an existing implementation, map it to this specification and report gaps before destructive changes.
- Work in small, reviewable changes. At the end of each milestone, run tests, linting, migrations and relevant health checks.
- Do not silently skip failing tests, weaken assertions, disable security controls or mark incomplete features as complete.
- Never commit secrets, real API keys, database passwords, private user data or .env files. Provide a safe .env.example.
- Do not store provider secrets in ordinary project JSON, logs, traces, frontend code or plaintext database columns.
- Keep business logic in service/domain modules, not Django views or Angular components.
- Document assumptions and decisions. Ask the project owner only when a decision materially affects security, data integrity, scope or an irreversible choice.

2. Product scope and users

Target users

User type	Primary use
AI/software developer	Configure providers, test prompts and compare models.
AI/ML engineer	Create datasets, run evaluations and inspect model behaviour.
Technical team member	Monitor latency, failures, usage and estimated cost.
Workspace administrator	Manage members, roles, project access and settings.

Core product capabilities

- **Identity and tenancy:** registration, login, workspaces, memberships, roles and project-level data isolation.
- **Provider management:** configure cloud and local providers, validate connections, list models when supported and store credentials securely.
- **Model playground:** submit prompts, configure inference parameters, compare outputs side by side and retain prompt history.

- **Prompt management:** named prompts, immutable versions, system/user messages and reproducible experiment references.
- **Datasets:** create and import structured evaluation examples with inputs, optional context and optional reference answers.
- **Evaluation:** batch execution, configurable metrics, results, model/prompt comparisons and report export.
- **Observability:** execution records, traces, latency, token usage, provider errors, estimated costs and searchable filters.
- **AI security:** controlled test cases for prompt injection, PII exposure and unsafe output, with findings and severity.
- **Analytics:** workspace/project summaries, trends, evaluation history and operational reports.

Out of scope for initial release

- Training foundation models from scratch or operating a public GPU inference service.
- Guaranteeing factual correctness or treating an LLM judge as ground truth.
- Automatically deploying customer applications or executing arbitrary agent tools.
- Billing customers, subscription management or public multi-tenant SaaS onboarding.
- A production-grade distributed microservice architecture.

3. Required technology decisions

Layer	Required choice	Implementation notes
Frontend	Angular + Tailwind CSS	Typed API services, lazy routes, reactive forms and reusable components.
Backend	Python + Django REST Framework	Versioned REST API; modular Django apps; service layer.
Database	PostgreSQL	Relational source of truth; UUID primary keys; migrations and indexes.
Async jobs	Celery + Redis	Evaluation runs, imports and scheduled aggregation.
AI providers	Provider adapter interface	OpenAI-compatible APIs, Hugging Face, LM Studio and Ollama.
Evaluation	DeepEval integration	Wrap in an internal metric registry; retain metric/version metadata.
Telemetry	OpenTelemetry	Trace model calls and background work; privacy-aware capture.
Deployment	Docker Compose	Frontend, API, database, Redis, worker and scheduler.
Testing	pytest/Django, Angular tests, Playwright	Unit, integration, permission, end-to-end and security tests.

4. System architecture

Use a modular monolith. The Django API owns authentication, tenancy, business rules and persistence. Celery workers execute long-running or retryable work. A provider adapter boundary isolates vendor-specific APIs. PostgreSQL is the source of truth; Redis is a broker/cache and must not become the only store for important results.

Request and evaluation flow

- User submits an execution or evaluation request through Angular.
- Django authenticates the user, checks workspace/project permissions, validates input and creates a durable record.
- For a long-running evaluation, Django enqueues a Celery task and returns a job/run identifier.
- The worker loads the run configuration, invokes the selected provider adapter and records execution metadata.
- Evaluation metrics are calculated by registered evaluators; each result records metric name, metric version, configuration and score.

- Trace and operational data are recorded with a correlation identifier. Sensitive content capture follows project privacy settings.
- The frontend retrieves status and results through the API. Use polling initially; consider server-sent events only after the core workflow is stable.

Provider abstraction: business services must call a common internal interface, not vendor SDKs directly. Each adapter translates the internal request into provider-specific calls and normalises responses.

```
ProviderAdapter
  validate_connection() -> ConnectionStatus
  list_models() -> list[ModelInfo]        # capability-dependent
  generate(request) -> GenerationResult
  capabilities() -> ProviderCapabilities
```

Normalised generation results should include provider/model identifiers, content (subject to capture policy), input/output token counts when supplied, latency, finish reason, request ID where available, and structured error information. Preserve raw provider metadata only when safe and useful.

5. Backend repository structure

```
backend/
  config/
    settings/
    urls.py
    celery.py
  apps/
    accounts/
    workspaces/
    projects/
    providers/
    models_registry/
    prompts/
    playground/
    datasets/
    evaluations/
    observability/
    security_scans/
    analytics/
  integrations/
    providers/
    evaluation/
    telemetry/
  common/
    permissions/
    exceptions/
    pagination/
    encryption/
    audit/
  tests/
  manage.py
```

Each Django app should separate models, serializers, selectors/query services where useful, business services, API views/viewsets, permissions, tasks and tests. Avoid an overly abstract generic framework. Keep abstractions driven by real repeated needs.

6. Frontend repository structure

```
frontend/src/app/
  core/
    authentication/
    guards/
    interceptors/
    layout/
  shared/
    components/
    pipes/
    models/
  features/
    dashboard/
    workspaces/
    projects/
    providers/
    playground/
    prompts/
    datasets/
    evaluations/
    observability/
    security/
    settings/
  app.routes.ts
```

Use feature-based organisation, strict TypeScript settings, typed request/response models and a centralised API layer. Components should focus on presentation and user interaction. Put API access and state coordination in feature services. Protect routes and handle expired authentication consistently.

7. Data model and integrity

Entity	Purpose / key relationships
User	Identity and authentication profile.
Workspace	Tenant boundary; owner and settings.
Membership	User-to-workspace relation and role.
Project	Belongs to a workspace; contains configurations and runs.
ProviderConfig	Provider type, endpoint, capability/configuration metadata.
ProviderSecret	Encrypted credential material, separated from ordinary config.
ModelConfig	Provider reference, model identifier and inference defaults.
Prompt / PromptVersion	Named prompt and immutable, numbered versions.
Dataset / DatasetItem	Evaluation dataset and structured examples.
EvaluationRun	Dataset, selected model/prompt versions, status and timestamps.
EvaluationResult	Per-item/per-metric score, explanation and evaluator metadata.
Trace / TraceEvent	Execution lifecycle, spans/events, timings and safe metadata.
SecurityScan / Finding	Test run, finding category, severity and evidence reference.
AuditEvent	Security-sensitive administrative and configuration actions.

- Use UUID primary keys and UTC timestamps. Add created_at/updated_at where appropriate.
- Use foreign keys and explicit on-delete behaviour. Do not rely on application code alone for referential integrity.

- Index common filters: workspace_id, project_id, created_at, status, provider/model and trace/run identifiers.
- Enforce tenant isolation in every query and mutation. A user-supplied object ID is never proof of access.
- Use transactions for multi-record state transitions. Define valid run status transitions and prevent invalid updates.
- Keep prompt versions immutable. A new edit creates a new version.
- Define retention and deletion behaviour for traces, prompt content, datasets, credentials and evaluation records.
- Do not store large binary files directly in relational rows; use a controlled file-storage abstraction if imports require it.

8. API contract principles

Use the base path **/api/v1/**. Publish an OpenAPI schema and keep it aligned with actual serializers and response behaviour. Use consistent pagination, filtering, validation errors and status codes.

Route family	Representative endpoints
Authentication	/auth/register, /auth/login, /auth/refresh, /auth/me
Workspaces	/workspaces, /workspaces/{id}/members
Projects	/projects
Providers	/projects/{id}/providers, /providers/{id}/test
Models	/providers/{id}/models
Playground	/projects/{id}/playground/execute
Prompts	/projects/{id}/prompts, /prompts/{id}/versions
Datasets	/projects/{id}/datasets, /datasets/{id}/items
Evaluations	/projects/{id}/evaluations, /evaluations/{id}
Observability	/projects/{id}/traces, /traces/{id}
Analytics	/projects/{id}/analytics/overview
Security	/projects/{id}/security/scans

Long-running operations must return a durable run/job ID and an initial status (for example queued). Provide status/result endpoints. Use idempotency protection for operations where retries could create duplicate runs. Never hold an HTTP request open while an entire dataset evaluation executes.

9. Evaluation engine requirements

Implement an internal evaluation orchestration service and metric registry. DeepEval is an integration, not the domain model. The platform must be able to add custom evaluators without changing the orchestration workflow.

- Initial metrics: answer relevance, faithfulness, factual correctness against reference, context precision, context recall, toxicity and custom metric hooks.
- Each metric has a stable key, display name, description, required inputs, configuration schema and version.
- A result records score, pass/fail or threshold status where applicable, explanation/evidence, evaluator/model version and execution timestamp.
- Represent unsupported or unscorable cases explicitly; do not convert missing scores to zero.
- Record the judge model and judge prompt/version when an LLM judge is used.
- Allow human review/annotation for sampled results and disagreement analysis.

- Make evaluation runs reproducible by snapshotting dataset version, prompt version, model/provider configuration and metric configuration.
- Control concurrency, retries, timeouts and per-provider rate limits. Retry transient errors only; do not blindly retry invalid requests.

10. Observability and privacy

Use OpenTelemetry-compatible traces and spans for request and model execution lifecycles. Persist the product's queryable execution records in PostgreSQL. Do not assume OpenTelemetry itself is the product database.

Minimum trace attributes: project, provider, model, operation, status, duration, token counts when available, error category, correlation ID and timestamps. Avoid placing secrets or unrestricted user content in span attributes.

- Make prompt/response content capture configurable per project, with a safe default that minimises content collection.
- Redact configured PII and secrets before persistence or export. Document that redaction is best-effort, not perfect.
- Record whether token counts and cost are provider-reported, estimated or unavailable.
- Maintain a configurable provider pricing catalogue with source and effective date. If no verified rate exists, show usage without a cost estimate.
- Apply retention controls and access permissions to trace detail and exports.
- Use structured logs with correlation IDs; never log passwords, access tokens, API keys or encryption keys.

11. Security requirements

- Use Django's secure password hashing (Argon2 where configured) and short-lived access tokens with refresh-token rotation/revocation strategy.
- Enforce workspace/project permissions server-side on every object access. Add tests for cross-tenant IDOR attempts.
- Encrypt provider credentials with authenticated encryption. Keep the encryption key outside the database and source control; document key rotation.
- Validate provider endpoint URLs and block loopback/private/link-local destinations unless explicitly configured as trusted local inference targets. Defend against SSRF and DNS rebinding where feasible.
- For local model connections, allow only approved host/network destinations. Never expose an unrestricted URL-fetch feature.
- Apply request size limits, rate limits, input validation, CORS restrictions and secure production settings.
- Treat model output and retrieved content as untrusted data. Do not execute generated code or tool instructions.
- Security scans must be controlled tests with bounded payloads, safe execution and clear consent/scope.
- Maintain audit records for credential changes, membership/role changes and sensitive administrative actions.

12. Docker and environment

Provide Dockerfiles and a Docker Compose configuration for the frontend, Django API, PostgreSQL, Redis, Celery worker and Celery Beat. Include health checks, named persistent volumes and a clear local development workflow.

The API container must be able to reach local inference services running on the host. Document platform-specific host addressing (for example host.docker.internal where supported) and allow an explicit configured endpoint. Do not assume that localhost inside a container refers to the host.

Use .env.example with placeholder values. Do not put production secrets in Compose files. Expose only the ports needed for local development; do not publish database or Redis ports in a production profile.

```
# Expected developer workflow (document exact commands)
docker compose up --build
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py createsuperuser
docker compose run --rm backend pytest
```

```
docker compose down
```

13. Implementation roadmap and gates

Milestone	Build	Exit criteria
0. Discovery	Inspect repo, write implementation plan, record assumptions.	Owner approves plan; no destructive changes.
1. Foundation	Repo, Docker, Django, Angular, PostgreSQL, Redis, Celery, CI.	All services healthy; migrations and smoke tests pass.
2. Identity/tenancy	Auth, workspaces, memberships, projects and RBAC.	Permission and cross-tenant tests pass.
3. Provider layer	Provider configuration, encrypted secrets, adapters and connection tests.	Mocked contracts and one local provider work end-to-end.
4. Playground	Prompt execution, comparison, history and prompt versions.	Same input can be run against two models and compared.
5. Datasets/evaluation	Dataset CRUD/import, metric registry, Celery runs and results.	Repeatable evaluation with persisted per-item results.
6. Observability	Traces, logs, latency, token usage, error and cost estimates.	Traces links to model execution; unavailable values remain explicit.
7. Security	PII checks, injection tests, unsafe output checks and controls.	Controlled scan and report workflow with tests.
8. Analytics/reports	Dashboard, filters, comparisons and export.	Dashboard numbers reconcile with stored source records.
9. Hardening	Performance, security review, docs, backups and deployment profile.	Clean deployment and full regression suite pass.

14. Testing and definition of done

- **Backend:** unit tests for services/metrics; integration tests for database and adapters; API tests for validation, auth and permissions.
- **Frontend:** component/service tests, form validation, route guard and error-state tests.
- **End-to-end:** register/login → create workspace/project → connect provider → execute prompt → run evaluation → inspect trace.
- **Security:** cross-workspace access tests, secret-redaction tests, SSRF allow/deny tests, rate-limit tests and safe scan tests.
- **Reliability:** worker restart, retry handling, task failure, provider timeout and duplicate submission tests.
- **Quality:** lint, type checks, migrations check, dependency audit and reproducible Docker startup.

A feature is done only when its acceptance criteria are met, tests pass, error states are handled, permissions are enforced, documentation is updated and the feature works in the Docker environment. A UI mock with no working backend is not a completed feature.

15. Required documentation and handover

- README with product purpose, architecture diagram, prerequisites and exact setup commands.
- Architecture Decision Records (ADRs) for material decisions and deviations.
- OpenAPI documentation and example requests/responses.
- Database relationship diagram and migration notes.
- Provider setup guides for cloud APIs, LM Studio and Ollama.
- Evaluation metric definitions, limitations and reproducibility notes.
- Security model, secret handling, privacy defaults and retention policy.
- Testing guide, troubleshooting guide and known limitations.

- Milestone status: completed, partial, blocked; include test commands and results.

16. First task for the coding agent

Do not immediately generate all product features. Begin with a repository audit and a concise implementation plan.

- List the existing files, framework versions, current working features and test status.
- Identify whether the repository is empty or already contains application code.
- Propose the exact initial directory structure and Docker services.
- Identify decisions that require owner approval; otherwise use this specification and sensible documented defaults.
- Implement Milestone 1 only: Docker-first foundation with Django REST, Angular, PostgreSQL, Redis and Celery.
- Add health checks, environment examples, migrations, smoke tests and setup documentation.
- Run the full Milestone 1 verification and report exact commands, outcomes and any limitations.
- Stop after Milestone 1 and wait for review before beginning Milestone 2.

The first milestone is intentionally limited. A reliable foundation is more valuable than a large volume of unverified generated code.

17. Acceptance checklist for the project owner

- ■ The agent has inspected the repository before editing.
- ■ The agent has not committed or exposed credentials.
- ■ Docker Compose starts all required foundation services.
- ■ Django health endpoint responds and database connectivity is verified.
- ■ Angular application builds and can reach the API.
- ■ Redis and Celery worker connectivity is verified with a test task.
- ■ Database migrations apply cleanly.
- ■ Backend and frontend smoke tests pass.
- ■ README and .env.example are complete.
- ■ The agent has stopped at Milestone 1 and supplied a verifiable report.

End of specification. This document is the baseline implementation contract for ModelScope AI. Any proposed deviation must include the reason, impact, alternatives considered and a documented decision.